

Instituto Tecnológico de Buenos Aires

Bases de Datos II

Trabajo Práctico Obligatorio

VetSalud

Grupo 7

Integrantes:

- Buela, Mateo - 64680
- Medina, Sol - 63577
- Rodriguez Acevedo, Clara - 66527

Fecha de entrega: 12 junio 2026

Índice

1. Introducción	1
2. Decisiones de Bases de Datos	2
2.1. Elección de MongoDB	2
2.2. Elección de Redis	2
2.3. División de responsabilidades entre motores	3
3. Modelo de Datos	4
3.1. Carga y extensión de los datasets	4
3.2. Modelo en MongoDB	4
3.3. Modelo en Redis	7
3.4. Relación entre ambos motores	8
4. Implementación de las Queries	9
4.1. Queries resueltas con MongoDB	9
4.1.1. Query 1: Pacientes activos con datos de propietario	9
4.1.2. Query 2: Consultas en seguimiento con veterinario y costo	9
4.1.3. Query 3: Historial completo de un paciente	10
4.1.4. Query 4: Propietarios con más de un paciente	12
4.1.5. Query 6: Pacientes con vacunas vencidas	13
4.1.6. Query 9: Consultas de control con costo menor a \$5.000	14
4.1.7. Query 10: Pacientes de una sucursal determinada	15
4.1.8. Query 12: Propietarios a revisar	15
4.1.9. Query 13: ABM de propietarios	16
4.2. Queries resueltas con Redis	17
4.2.1. Query 15: Decremento de stock tras una consulta	17
4.3. Queries que combinan MongoDB y Redis	18
4.3.1. Query 8: Stock de productos con menos de 50 unidades	18
4.3.2. Query 14: Registro de nueva consulta médica	19
4.4. Queries cacheadas	19
4.4.1. Query 5: Veterinarios activos con consultas en los últimos 60 días	20
4.4.2. Query 7: Top 5 diagnósticos más frecuentes	21
4.4.3. Query 11: Ingresos por veterinario en el mes actual	21
5. Limitaciones	23
6. Conclusiones	24

1 Introducción

VetSalud es una red ficticia de clínicas veterinarias en Argentina. Este trabajo monta un sistema de gestión sobre una arquitectura de persistencia políglota con dos motores NoSQL. El sistema administra propietarios, sus mascotas o pacientes, veterinarios, consultas y cirugías, vacunaciones, y el stock farmacéutico. Las bases corren en contenedores Docker, y sobre ellas construimos una API REST en Node con Express junto con un dashboard web para ejecutar y visualizar las consultas.

Más allá de implementar las consultas, el foco del trabajo está en justificar cómo se reparten las responsabilidades entre ambos motores y qué aporta cada uno. MongoDB concentra el dominio persistente y estructurado de la clínica, mientras que Redis queda reservado para información de acceso rápido, contadores y resultados derivados. Las próximas secciones desarrollan esas decisiones, el modelo de datos que surge de ellas y la implementación de cada una de las quince consultas.

2 Decisiones de Bases de Datos

2.1 Elección de MongoDB

Elegimos MongoDB como base principal del dominio clínico y administrativo del sistema. La decisión se apoyó en las características del dominio y en el tipo de consultas que debíamos resolver.

- **Flexibilidad de esquema.** El dominio veterinario puede cambiar con el tiempo, y MongoDB permite agregar campos nuevos sin rediseñar todo el esquema.
- **Datos con estructura de documento.** Las entidades principales del sistema tienen varios atributos propios y se representan naturalmente como documentos. Propietarios, pacientes, veterinarios, consultas, vacunaciones y productos son registros con estructuras complejas.
- **Consultas analíticas sobre el dominio.** Muchas queries requieren filtrar, cruzar entidades, agrupar y proyectar resultados, por lo que utilizamos el Aggregation Framework.
- **Atomicidad por documento.** Las operaciones principales del sistema afectan un documento central por vez, como el alta de un propietario, la modificación de sus datos, la baja lógica de un propietario o la inserción de una consulta. MongoDB garantiza atomicidad sobre cada documento, lo cual alcanza para la mayoría de las operaciones del dominio clínico.
- **Consistencia para datos clínicos y administrativos.** Nuestra prioridad era no perder ni duplicar registros. Por eso usamos MongoDB como fuente principal del dominio persistente y centralizamos allí las escrituras.

2.2 Elección de Redis

Elegimos Redis para resolver las partes del sistema que necesitaban acceso rápido, operaciones atómicas simples y expiración automática. Redis cumple dos roles dentro del sistema. Por un lado administra las unidades disponibles del stock farmacéutico. Por otro, cachea resultados de consultas pesadas.

- **Stock como contador frecuente.** Las unidades disponibles de cada producto cambian cada vez que una consulta consume insumos. Este dato se comporta más como un contador operativo que como un documento clínico, por lo que Redis resulta más adecuado que MongoDB para administrarlo.
- **Lectura rápida de stock puntual.** La lógica de stock necesita consultar cuántas unidades quedan de un producto específico. Redis permite resolver esta lectura de forma directa y con muy baja latencia.
- **Consulta eficiente de stock bajo.** También necesitamos listar productos cuyo stock esté por debajo de cierto umbral. Para eso usamos un Sorted Set, donde el identificador del producto es el valor y la cantidad disponible es el score.
- **Decremento atómico.** Cuando una consulta consume productos, el decremento individual del contador debe ejecutarse de forma indivisible para evitar resultados negativos. Redis permite validar el stock disponible y descontar unidades de forma atómica mediante un script Lua.
- **Caché con TTL.** Algunas queries del dashboard agregan sobre toda la colección de consultas y pueden ejecutarse muchas veces. Redis permite guardar estos resultados con un TTL corto, evitando recalcular agregaciones costosas en cada lectura.

2.3 División de responsabilidades entre motores

La arquitectura quedó dividida según el tipo de dato y el tipo de operación. MongoDB concentra la información persistente y descriptiva del sistema, mientras que Redis almacena datos operativos o derivados que requieren acceso rápido.

- **MongoDB como fuente principal del dominio.** Propietarios, pacientes, veterinarios, consultas, vacunaciones y productos viven en MongoDB. Estos datos representan el estado clínico y administrativo de VetSalud.
- **Redis como fuente de verdad del stock.** Las unidades disponibles de cada producto viven en Redis. El catálogo del producto está en MongoDB, pero la cantidad disponible se administra en Redis porque cambia con más frecuencia y requiere operaciones atómicas.
- **Redis como caché descartable.** Las claves `cache:*` guardan resultados derivados de consultas sobre MongoDB. Como no son información primaria, si una clave expira o se invalida el resultado puede recalcularse.
- **Separación por prefijos.** Las claves de stock usan el prefijo `stock:`, mientras que las claves de caché usan `cache:`. Esto evita mezclar datos con semánticas distintas dentro del mismo motor.
- **Limitación cross-motor.** MongoDB y Redis no comparten una transacción global, lo que genera una posible ventana de inconsistencia en operaciones que tocan ambos motores. La sección de limitaciones detalla cómo se mitiga.

3 Modelo de Datos

El modelo de datos se diseñó pensando en las consultas que debía resolver el sistema. El modelo reparte la información según el tipo de dato y el tipo de operación esperada, en lugar de copiar un modelo relacional dentro de una base NoSQL. MongoDB concentra las entidades clínicas y administrativas, mientras que Redis guarda estructuras específicas para stock y caché.

El sistema utiliza la base MongoDB `vetsalud`. Durante la carga inicial, el script de seed lee los archivos CSV, transforma los tipos de datos necesarios y carga las colecciones desde cero. Los identificadores provistos por los datasets se conservan como `_id` de cada documento.

3.1 Carga y extensión de los datasets

Los archivos ubicados en `data/` parten de los CSV de base, extendidos con registros propios. La carga se realiza desde esos CSV extendidos mediante `npm run seed`.

Dataset	Registros base	Registros usados	Agregados
<code>propietarios.csv</code>	6	18	12
<code>pacientes.csv</code>	8	22	14
<code>veterinarios.csv</code>	5	15	10
<code>consultas.csv</code>	8	29	21
<code>vacunaciones.csv</code>	6	33	27
<code>stock_farmaceutico.csv</code>	6	20	14

Cuadro 1: Comparación entre los datasets de base y los datasets extendidos usados por el sistema.

Además de agregar filas, se incorporaron dos campos al modelo cargado desde CSV. En `propietarios.csv` se agregó `activo`, necesario para implementar la baja lógica. En `consultas.csv` se agregó `tipo`, usado para distinguir entre `Consulta` y `Cirugia`. El seed también agrega algunas consultas con fechas relativas al mes actual. Estas consultas se suman a los CSV extendidos y ayudan a que la vista de ingresos mensuales tenga datos representativos aunque el sistema se ejecute en otro mes.

3.2 Modelo en MongoDB

En MongoDB se guardan las entidades principales del dominio de VetSalud. Cada una se representa como una colección independiente porque tiene identidad propia y puede ser consultada desde distintos puntos del sistema.

Las colecciones principales son:

- `propietarios`
- `pacientes`
- `veterinarios`
- `consultas`
- `vacunaciones`
- `productos`

La decisión general fue mantener las relaciones mediante referencias por id. Por ejemplo, un paciente referencia a su propietario con `id_propietario`, y una consulta referencia tanto al paciente como al veterinario mediante `id_paciente` e `id_vet`. Esto permite mantener las entidades separadas y evitar duplicar información descriptiva, pero al mismo tiempo deja disponible la posibilidad de cruzarlas mediante `$lookup` cuando una query necesita información combinada.

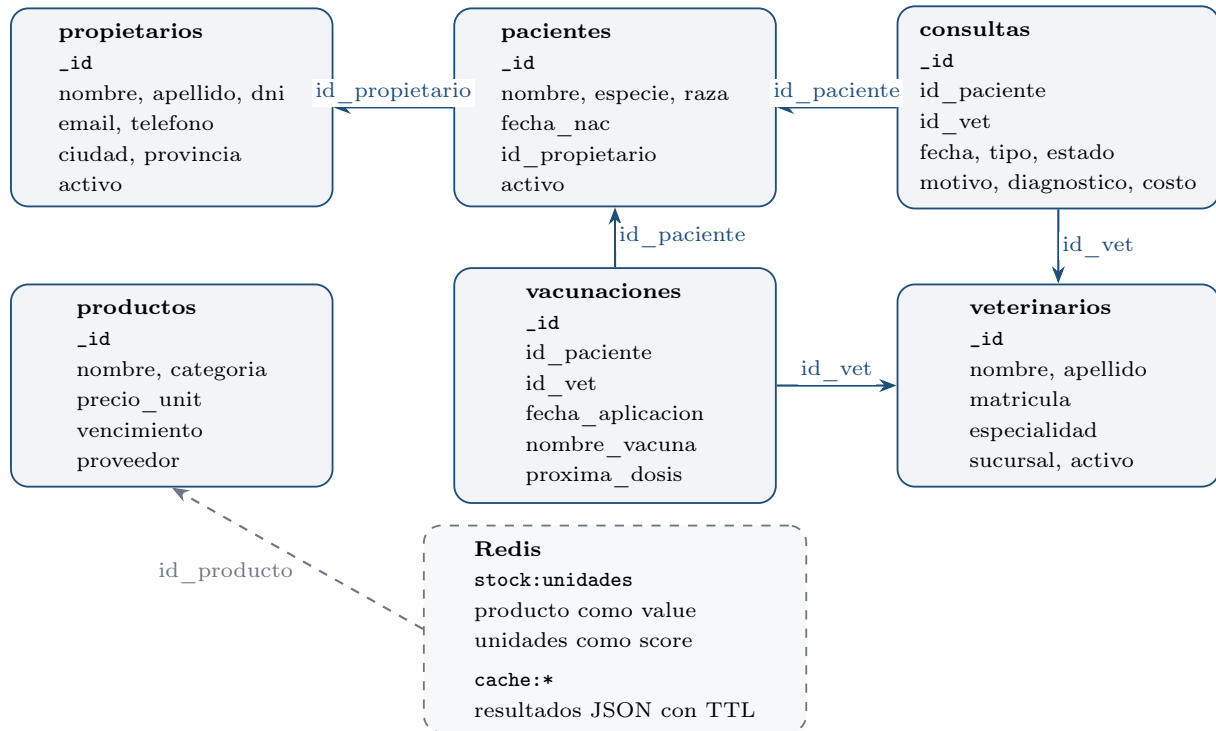


Figura 1: Resumen UML simplificado del modelo de datos y sus referencias principales.

Los propietarios contienen la información de contacto y ubicación de cada cliente de la clínica. Además, incluyen un campo `activo`, utilizado para implementar baja lógica. Esta decisión permite conservar los datos históricos aunque un propietario deje de estar activo.

Ejemplo de propietario

```

1  {
2    "_id": "C001",
3    "nombre": "Ana",
4    "apellido": "Rodriguez",
5    "dni": "30456789",
6    "email": "ana@gmail.com",
7    "telefono": "1145001234",
8    "ciudad": "Buenos Aires",
9    "provincia": "Buenos Aires",
10   "activo": true
11 }
  
```

Los pacientes representan las mascotas atendidas por VetSalud. Cada paciente referencia a su propietario, pero no se encuentra embebido dentro del documento del dueño. Elegimos esta separación porque un propietario puede tener varios pacientes y porque muchas consultas necesitan trabajar directamente sobre la colección de pacientes. El campo `fecha_nac` se carga como fecha en MongoDB.

Ejemplo de paciente

```
1 {
2   "_id": "P001",
3   "nombre": "Firulais",
4   "especie": "Perro",
5   "raza": "Labrador",
6   "fecha_nac": "2019-04-12T00:00:00.000Z",
7   "id_propietario": "C001",
8   "activo": true
9 }
```

Los veterinarios se modelan como una colección propia porque son una entidad central para varias consultas del sistema. Además de los datos profesionales, cada veterinario tiene una sucursal asociada. Esta decisión es importante porque algunas queries necesitan vincular pacientes o atenciones con la sucursal del veterinario que las realizó.

Ejemplo de veterinario

```
1 {
2   "_id": "V001",
3   "nombre": "Gabriela",
4   "apellido": "Fontana",
5   "matricula": "VET-0041",
6   "especialidad": "Clinica General",
7   "sucursal": "Palermo",
8   "activo": true
9 }
```

Una de las decisiones más importantes del modelo fue guardar consultas y cirugías en una misma colección llamada `consultas`. Ambas representan atenciones clínicas y comparten la mayor parte de sus campos, como paciente, veterinario, fecha, motivo, diagnóstico, costo y estado. Para diferenciarlas se utiliza el campo `tipo`. Esto evita duplicar pipelines y simplifica las queries que trabajan sobre historial clínico, diagnósticos frecuentes o ingresos por veterinario.

Ejemplo de consulta

```
1 {
2   "_id": "CON001",
3   "id_paciente": "P001",
4   "id_vet": "V001",
5   "fecha": "2025-10-05T00:00:00.000Z",
6   "tipo": "Consulta",
7   "motivo": "Control anual",
8   "diagnostico": "Sano",
9   "costo": 4500,
10  "estado": "Cerrada"
11 }
```

Las vacunaciones se guardan en una colección separada. Aunque también se relacionan con pa-

cientes y veterinarios, su estructura tiene otra semántica, ya que lo central es la vacuna aplicada y la fecha de próxima dosis. Mezclarlas con consultas habría generado documentos con campos opcionales sin una ventaja clara para las queries que debíamos resolver. Los campos `fecha_aplicacion` y `proxima_dosis` se cargan como fechas.

Ejemplo de vacunación

```
1 {
2   "_id": "VAC001",
3   "id_paciente": "P001",
4   "id_vet": "V001",
5   "fecha_aplicacion": "2025-10-05T00:00:00.000Z",
6   "nombre_vacuna": "Antirrabica",
7   "proxima_dosis": "2026-10-05T00:00:00.000Z"
8 }
```

Por último, la colección `productos` guarda el catálogo farmacéutico. En MongoDB se conserva la información descriptiva y relativamente estable de cada producto, como nombre, categoría, precio unitario, vencimiento y proveedor. La cantidad disponible no se guarda en esta colección porque forma parte del stock operativo, que cambia con mayor frecuencia y se administra en Redis.

Ejemplo de producto

```
1 {
2   "_id": "PRD001",
3   "nombre": "Amoxicilina 250mg",
4   "categoria": "Antibiotico",
5   "precio_unit": 850,
6   "vencimiento": "2026-06-30T00:00:00.000Z",
7   "proveedor": "VetFarma SA"
8 }
```

Además de las colecciones, se definieron índices sobre campos usados frecuentemente en filtros, relaciones o rangos. El seed crea índices sobre `fecha` e `id_vet` en `consultas`, sobre `id_propietario` en `pacientes`, y sobre `proveedor` y `vencimiento` en `productos`. Estos índices acompañan las consultas implementadas y evitan depender únicamente de recorridos completos sobre las colecciones.

También se define una vista formal para la Query 11, llamada `vista_ingresos_vet_mes`. Esta vista usa un pipeline de agregación sobre consultas, cruza con veterinarios y expone los ingresos del mes actual agrupados por profesional.

3.3 Modelo en Redis

Redis se usa con un modelo más acotado que MongoDB. No se intentó replicar las entidades clínicas ni duplicar el dominio completo. Solo se guardan estructuras donde Redis aporta una ventaja clara, que son las unidades disponibles del stock farmacéutico y la caché de consultas pesadas.

La estructura principal es `stock:unidades`, un Sorted Set donde cada producto tiene como valor su identificador y como score la cantidad disponible. Esta estructura se carga desde el campo `unidades` del CSV de stock farmacéutico.

Estructura de stock en Redis

```
1 stock:unidades
2
3 value    score
4 PRD001   120
5 PRD004   40
6 PRD006   25
7 PRD017   15
```

El uso de un Sorted Set responde directamente a la necesidad de ordenar y filtrar por cantidad disponible. Para la consulta de stock bajo, Redis permite buscar por rango sobre el score y obtener rápidamente los productos por debajo de un umbral.

Consulta de stock bajo

```
1 ZRANGEBYSCORE stock:unidades -inf 49
```

El descuento de stock es el caso más sensible porque involucra concurrencia sobre un contador compartido. Por eso, la lógica de decremento se ejecuta dentro de Redis mediante un script Lua atómico, que verifica existencia, valida cantidad suficiente y descuenta en una única operación indivisible.

Además del stock, Redis guarda resultados cacheados de algunas queries del dashboard. Estas claves siguen el prefijo `cache:` y contienen resultados serializados en JSON con un TTL corto. Esto permite acelerar consultas frecuentes sin convertir esos resultados en fuente de verdad.

Las claves de caché usadas por el sistema son:

- `cache:vets-consultas-60d`, con TTL de 300 segundos.
- `cache:top-diagnosticos`, con TTL de 300 segundos.
- `cache:ingresos-vet-mes`, con TTL de 60 segundos.

Estas claves representan información derivada. Si expiran o se invalidan, el resultado puede recalcularse desde MongoDB. Por eso tienen una semántica distinta a la de `stock:unidades`. El stock representa estado operativo del sistema, mientras que la caché representa resultados temporales.

3.4 Relación entre ambos motores

La relación entre MongoDB y Redis aparece principalmente en el manejo del stock farmacéutico. MongoDB guarda el catálogo de productos, mientras que Redis guarda las unidades disponibles. Esto significa que para mostrar productos con stock bajo el sistema combina información de ambos motores. Redis identifica rápidamente qué productos están por debajo del umbral y MongoDB aporta los datos descriptivos de esos productos.

Este diseño evita duplicar información descriptiva. Redis no necesita saber el nombre, proveedor o vencimiento de cada producto, solo necesita saber cuántas unidades quedan. MongoDB, por su parte, no necesita actualizar constantemente un contador de stock que cambia con cada consulta. Cada motor conserva la parte del dato que mejor se adapta a su modelo.

La relación también aparece en las queries cacheadas. En esos casos, MongoDB sigue siendo la fuente de verdad, ya que la query se calcula sobre sus colecciones y el resultado se guarda temporalmente en Redis. Redis funciona como una capa de aceleración para lecturas frecuentes.

4 Implementación de las Queries

4.1 Queries resueltas con MongoDB

4.1.1 Query 1: Pacientes activos con datos de propietario

La primera query expone el endpoint `GET /api/pacientes-activos` y resuelve el listado de pacientes activos junto con los datos completos de su propietario. Se implementa sobre MongoDB utilizando las colecciones `pacientes` y `propietarios`.

La consulta primero filtra los documentos de `pacientes` cuyo campo `activo` vale `true`. Luego utiliza `$lookup` para unir cada paciente con su propietario, comparando `pacientes.id_propietario` contra `propietarios._id`. Como cada paciente tiene un único propietario, se aplica `$unwind` sobre el resultado de la unión y finalmente se proyectan los campos relevantes.

La proyección incluye los datos del paciente activo junto con todos los campos del propietario cargados por el seed, como identificador, nombre, apellido, DNI, email, teléfono, ciudad, provincia y estado lógico.

Ejemplo de resultado de Query 1

```
1  [
2    {
3      "_id": "P005",
4      "nombre": "Rex",
5      "especie": "Perro",
6      "raza": "Pastor Aleman",
7      "fecha_nac": "2020-09-08T00:00:00.000Z",
8      "activo": true,
9      "propietario": {
10       "_id": "C004",
11       "nombre": "Diego",
12       "apellido": "Herrera",
13       "dni": "35789012",
14       "email": "diego@gmail.com",
15       "telefono": "1178004567",
16       "ciudad": "Mendoza",
17       "provincia": "Mendoza",
18       "activo": true
19     }
20  }
21 ]
```

4.1.2 Query 2: Consultas en seguimiento con veterinario y costo

La segunda query expone el endpoint `GET /api/consultas-seguimiento` y lista las consultas médicas abiertas cuyo estado es `Seguimiento`. Se resuelve íntegramente en MongoDB usando las colecciones `consultas`, `veterinarios` y `pacientes`.

El pipeline comienza con un `$match` sobre `consultas.estado` para quedarse únicamente con las atenciones en seguimiento. Luego realiza un `$lookup` contra `veterinarios` usando `id_vet`, lo que permite mostrar el veterinario asignado a la atención. Después hace un segundo `$lookup` contra `pacientes`

para incorporar datos básicos del animal atendido. En ambos casos se usa `$unwind`, porque cada consulta referencia a un único veterinario y a un único paciente.

La proyección final conserva los datos principales de la consulta, incluyendo fecha, tipo, motivo, diagnóstico, estado y costo. También incluye el identificador del paciente, un bloque con datos básicos del paciente y otro bloque con los datos profesionales del veterinario. El resultado se ordena por fecha descendente para mostrar primero las atenciones más recientes que todavía requieren seguimiento.

Ejemplo de resultado de Query 2

```
1  [
2    {
3      "_id": "CON004",
4      "id_paciente": "P004",
5      "fecha": "2025-11-01T00:00:00.000Z",
6      "tipo": "Consulta",
7      "motivo": "Perdida de apetito",
8      "diagnostico": "Estres ambiental",
9      "costo": 3800,
10     "estado": "Seguimiento",
11     "paciente": {
12       "_id": "P004",
13       "nombre": "Nube",
14       "especie": "Conejo",
15       "raza": "Angora"
16     },
17     "veterinario": {
18       "_id": "V001",
19       "nombre": "Gabriela",
20       "apellido": "Fontana",
21       "matricula": "VET-0041",
22       "especialidad": "Clinica General",
23       "sucursal": "Palermo"
24     }
25   }
26 ]
```

4.1.3 Query 3: Historial completo de un paciente

La tercera query expone el endpoint `GET /api/historial-paciente/:id` y reconstruye el historial clínico de un paciente a partir de sus consultas y vacunaciones. Se resuelve principalmente con MongoDB, usando las colecciones `pacientes`, `consultas`, `vacunaciones` y `veterinarios`, y luego combina los resultados en Node.js.

Antes de ejecutar los pipelines, la función valida que el paciente solicitado exista. Si el identificador no corresponde a ningún documento de `pacientes`, se devuelve un error. Esta validación evita construir historiales sobre pacientes inexistentes.

La implementación ejecuta dos agregaciones en paralelo. La primera consulta la colección `consultas`, filtra por `id_paciente`, une cada atención con su veterinario mediante `$lookup` y proyecta el evento con un formato común. La segunda hace lo mismo sobre `vacunaciones`, usando `fecha_aplicacion` como fecha del evento e incorporando los campos propios de vacunas, como `nombre_vacuna` y `proxima_dosis`.

Se decidió realizar la combinación final en la capa de servicio porque el historial integra eventos de dos colecciones con estructuras distintas. MongoDB concentra el trabajo más pesado de cada origen, ya que filtra por paciente, resuelve la unión con veterinarios y proyecta cada evento con los campos necesarios. Node.js queda limitado a una tarea de orquestación, llevando ambas listas a un formato común, uniéndolas y aplicando el criterio de ordenamiento final.

Ambos resultados se normalizan a una misma estructura con campos como `tipo_evento`, `id_evento`, `fecha`, `paciente` y `veterinario`. Luego se concatenan en Node.js y se ordenan por fecha descendente, mostrando primero los eventos más recientes. En caso de empate de fecha, se usa el identificador del evento como segundo criterio de orden.

Fragmento de resultado de Query 3 para P001

```
1  [
2    {
3      "tipo_evento": "Consulta",
4      "id_evento": "CON007",
5      "fecha": "2025-12-02T00:00:00.000Z",
6      "paciente": {
7        "_id": "P001",
8        "nombre": "Firulais",
9        "especie": "Perro",
10       "raza": "Labrador"
11     },
12     "veterinario": {
13       "_id": "V001",
14       "nombre": "Gabriela",
15       "apellido": "Fontana",
16       "matricula": "VET-0041",
17       "sucursal": "Palermo"
18     },
19     "motivo": "Control post-vacuna",
20     "diagnostico": "Sin novedades",
21     "costo": 2000,
22     "estado": "Cerrada",
23     "nombre_vacuna": null,
24     "proxima_dosis": null
25   },
26   {
27     "tipo_evento": "Vacunacion",
28     "id_evento": "VAC001",
29     "fecha": "2025-10-05T00:00:00.000Z",
30     "paciente": {
31       "_id": "P001",
32       "nombre": "Firulais",
33       "especie": "Perro",
34       "raza": "Labrador"
35     },
36     "veterinario": {
37       "_id": "V001",
38       "nombre": "Gabriela",
```

```

39     "apellido": "Fontana",
40     "matricula": "VET-0041",
41     "sucursal": "Palermo"
42   },
43   "motivo": null,
44   "diagnostico": null,
45   "costo": null,
46   "estado": null,
47   "nombre_vacuna": "Antirrabica",
48   "proxima_dosis": "2026-10-05T00:00:00.000Z"
49 }
50 ]

```

4.1.4 Query 4: Propietarios con más de un paciente

La cuarta query expone el endpoint `GET /api/proprietarios-multiples-pacientes` y lista los propietarios que tienen más de un paciente registrado. Se implementa sobre MongoDB usando las colecciones `pacientes` y `proprietarios`.

El pipeline comienza desde `pacientes`, porque el dato que se necesita contar es la cantidad de mascotas asociadas a cada propietario. Primero agrupa por `id_propietario`, calcula `cantidad_pacientes` con `$sum` y guarda en un arreglo los datos básicos de cada paciente mediante `$push`. Luego aplica un `$match` para conservar únicamente los grupos con más de un paciente.

Después de identificar los propietarios que cumplen la condición, la query usa `$lookup` contra `proprietarios` para incorporar sus datos de contacto. Como cada grupo corresponde a un único propietario, se aplica `$unwind` y finalmente se proyecta una salida con el identificador del propietario, la cantidad total de pacientes, los datos del propietario y el detalle de las mascotas asociadas.

La consulta cuenta todos los pacientes registrados, incluidos los inactivos, ya que el requerimiento no restringe el conteo a mascotas activas. Por eso el resultado conserva el campo `activo` en cada paciente, lo que permite ver si alguna mascota asociada está dada de baja sin excluirla del conteo. El resultado se ordena por cantidad de pacientes en forma descendente y, ante empate, por identificador de propietario.

Ejemplo de resultado de Query 4

```

1  [
2    {
3      "cantidad_pacientes": 3,
4      "pacientes": [
5        {
6          "_id": "P001",
7          "nombre": "Firulais",
8          "especie": "Perro",
9          "raza": "Labrador",
10         "activo": true
11       },
12       {
13         "_id": "P003",
14         "nombre": "Coco",
15         "especie": "Perro",

```

```

16     "raza": "Beagle",
17     "activo": false
18 },
19 {
20     "_id": "P018",
21     "nombre": "Pelusa",
22     "especie": "Conejo",
23     "raza": "Holland Lop",
24     "activo": true
25 }
26 ],
27 "propietario": {
28     "_id": "C001",
29     "nombre": "Ana",
30     "apellido": "Rodriguez",
31     "email": "ana@gmail.com",
32     "telefono": "1145001234",
33     "ciudad": "Buenos Aires",
34     "provincia": "Buenos Aires",
35     "activo": true
36 },
37 "id_propietario": "C001"
38 }
39 ]

```

4.1.5 Query 6: Pacientes con vacunas vencidas

La sexta query expone el endpoint `GET /api/pacientes-vacunas-vencidas` y lista los pacientes que tienen al menos una vacunación cuya próxima dosis es anterior a la fecha actual. Se resuelve íntegramente en MongoDB usando las colecciones `vacunaciones` y `pacientes`.

El pipeline parte de `vacunaciones`, porque el criterio de vencimiento está en el campo `proxima_dosis`. La función toma la fecha del sistema, la normaliza al inicio del día y filtra con `$match` las vacunaciones donde `proxima_dosis` es anterior a hoy. Esta normalización evita marcar como vencida una vacuna cuya próxima dosis sea el mismo día de ejecución.

Después, la query agrupa por paciente. Para cada uno calcula la cantidad de vacunas vencidas, conserva el detalle en un arreglo y obtiene la próxima dosis más antigua con `$min`. Luego realiza un `$lookup` contra `pacientes` para incorporar los datos básicos del animal y proyecta el resultado final.

El ordenamiento usa primero `proxima_dosis_mas_antigua` en forma ascendente, de modo que los pacientes con vencimientos más antiguos aparezcan antes. Esto permite usar la consulta como una lista de priorización operativa para contactar propietarios o programar revacunaciones.

Ejemplo de resultado de Query 6

```

1  [
2    {
3      "cantidad_vacunas_vencidas": 2,
4      "vacunas_vencidas": [
5        {

```

```

6      "id_vacuna": "VAC022",
7      "nombre_vacuna": "Sextuple",
8      "fecha_aplicacion": "2024-06-10T00:00:00.000Z",
9      "proxima_dosis": "2025-06-10T00:00:00.000Z",
10     "id_vet": "V001"
11   },
12   {
13     "id_vacuna": "VAC031",
14     "nombre_vacuna": "Triple Felina",
15     "fecha_aplicacion": "2024-04-15T00:00:00.000Z",
16     "proxima_dosis": "2025-04-15T00:00:00.000Z",
17     "id_vet": "V001"
18   }
19 ],
20 "proxima_dosis_mas_antigua": "2025-04-15T00:00:00.000Z",
21 "paciente": {
22   "_id": "P003",
23   "nombre": "Coco",
24   "especie": "Perro",
25   "raza": "Beagle",
26   "activo": false
27 }
28 }
29 ]

```

4.1.6 Query 9: Consultas de control con costo menor a \$5.000

La novena query expone el endpoint `GET /api/controles-baratos?max=5000` y lista las consultas de control cuyo costo es menor al umbral indicado. Se resuelve con MongoDB usando las colecciones `consultas` y `pacientes`.

Para esta query asumimos que un control equivale a una consulta médica no quirúrgica, por lo que cualquier documento con `tipo` igual a `Consulta` cuenta como control.

El pipeline aplica un `$match` compuesto sobre `tipo` y `costo`. Luego hace un `$lookup` contra `pacientes` para mostrar el animal atendido, aplica `$unwind` porque cada consulta tiene un único paciente, ordena por costo ascendente y proyecta los datos principales de la atención.

Ejemplo de resultado de Query 9

```

1  [
2    {
3      "_id": "CON007",
4      "fecha": "2025-12-02T00:00:00.000Z",
5      "motivo": "Control post-vacuna",
6      "diagnostico": "Sin novedades",
7      "costo": 2000,
8      "paciente": {
9        "nombre": "Firulais",
10       "especie": "Perro"
11     }

```



```
12     }
13 ]
```

4.1.7 Query 10: Pacientes de una sucursal determinada

La décima query expone el endpoint de pacientes por sucursal y lista los pacientes asociados a una sucursal determinada. Se resuelve con MongoDB usando las colecciones `veterinarios`, `consultas`, `vacunaciones` y `pacientes`.

La sucursal es un atributo del veterinario y no del paciente. Por eso el pipeline comienza desde `veterinarios` y filtra por el nombre de sucursal recibido como parámetro. A partir de esos veterinarios, la query busca pacientes atendidos tanto en `consultas` como en `vacunaciones`.

Después de traer ambos conjuntos, se usa `$setUnion` para unir los identificadores de pacientes sin duplicados. Luego se desarma el arreglo, se agrupa nuevamente por paciente para deduplicar entre distintos veterinarios de la misma sucursal y se hace un `$lookup` final contra `pacientes` para devolver el documento completo del animal.

Fragmento de resultado de Query 10 para Palermo

```
1  [
2    {
3      "_id": "P001",
4      "nombre": "Firulais",
5      "especie": "Perro",
6      "raza": "Labrador",
7      "fecha_nac": "2019-04-12T00:00:00.000Z",
8      "id_propietario": "C001",
9      "activo": true
10   },
11   {
12     "_id": "P002",
13     "nombre": "Michi",
14     "especie": "Gato",
15     "raza": "Siames",
16     "fecha_nac": "2021-07-03T00:00:00.000Z",
17     "id_propietario": "C002",
18     "activo": true
19   }
20 ]
```

4.1.8 Query 12: Propietarios a revisar

La duodécima query expone el endpoint `GET /api/proprietarios-sin-consultas`. Para esta consulta consideramos como propietarios a revisar a los que no registran actividad clínica, ya sea porque no tienen mascotas registradas o porque tienen mascotas pero ninguna con consultas en el último año.

La función calcula el límite temporal a partir de la fecha del sistema, tomando el inicio del día ubicado un año atrás. Une cada dueño con sus pacientes mediante `$lookup` y, con esos identificadores de pacientes, busca consultas cuya fecha sea mayor o igual a ese límite.

Después agrega la cantidad de mascotas y la cantidad de consultas recientes. La condición final conserva propietarios sin mascotas o sin consultas en el período. Esta interpretación permite detectar dueños sin actividad reciente, ya sea por la ausencia de mascotas o por la falta de consultas durante el período analizado.

Para que el resultado sea fácil de leer, la query asigna una `categoria`. Si el propietario no tiene mascotas queda como `Sin mascotas`, y en caso contrario queda como `Sin consultas en el último año`. Ese mismo criterio se usa para ordenar el resultado. La consulta tiene en cuenta consultas y deja afuera las vacunaciones, dado que el criterio se define sobre consultas registradas.

Fragmento de resultado de Query 12

```
1  [
2    {
3      "_id": "C009",
4      "nombre": "Sofia",
5      "apellido": "Mendez",
6      "email": "sofi@gmail.com",
7      "ciudad": "San Miguel de Tucuman",
8      "provincia": "Tucuman",
9      "activo": true,
10     "cantidad_mascotas": 0,
11     "cantidad_consultas_recientes": 0,
12     "categoria": "Sin mascotas"
13   },
14   {
15     "_id": "C008",
16     "nombre": "Federico",
17     "apellido": "Russo",
18     "email": "fede@gmail.com",
19     "ciudad": "Mar del Plata",
20     "provincia": "Buenos Aires",
21     "activo": true,
22     "cantidad_mascotas": 1,
23     "cantidad_consultas_recientes": 0,
24     "categoria": "Sin consultas en el ultimo anio"
25   }
26 ]
```

4.1.9 Query 13: ABM de propietarios

La decimotercera query implementa el ABM completo de propietarios. Se resuelve sobre MongoDB usando la colección `propietarios` y expone tres endpoints:

- `POST /api/propietarios`
- `PUT /api/propietarios/:id`
- `DELETE /api/propietarios/:id`

El alta valida que el documento recibido tenga `_id` y que no exista otro propietario con ese mismo identificador. Si la validación pasa, inserta el documento recibido en `propietarios`. El campo

`activo` queda con valor `true` por defecto si no viene informado, pero se respeta el valor enviado en el alta.

La modificación toma el identificador desde la URL y aplica un `$set` únicamente con los campos enviados por el cliente. Antes de actualizar, se elimina `_id` del payload para impedir que una modificación cambie la clave primaria del documento. Si no existe un propietario con ese identificador, la función devuelve un error.

La baja se implementa como baja lógica. En lugar de borrar físicamente el documento, actualiza `activo` a `false`. Esto permite conservar trazabilidad y evita dejar referencias huérfanas desde pacientes, consultas o historiales que ya estaban asociados a ese propietario.

Ejemplos de respuesta de Query 13

```
1 {
2   "alta": {
3     "ok": true,
4     "_id": "C999"
5   },
6   "modificacion": {
7     "ok": true,
8     "modificados": 1
9   },
10  "baja_logica": {
11    "ok": true,
12    "baja_logica": "C999"
13  }
14 }
```

4.2 Queries resueltas con Redis

4.2.1 Query 15: Decremento de stock tras una consulta

La decimoquinta query expone el endpoint `POST /api/decrementar-stock` y descuenta unidades de un producto del stock farmacéutico. Se resuelve íntegramente en Redis usando la clave `stock:unidades`.

El stock está modelado como un Sorted Set, donde cada producto es un miembro del conjunto y la cantidad disponible se guarda como score. La función recibe `id_producto` y `cantidad`. Antes de ejecutar el descuento valida que el producto esté informado y que la cantidad sea un entero positivo.

El decremento se realiza mediante un script Lua ejecutado con `redis.eval`. Dentro de ese script Redis verifica que el producto exista, valida que el stock actual alcance para cubrir la cantidad solicitada y descuenta las unidades con `ZINCRBY`. Como el script se ejecuta de forma atómica, dos requests concurrentes no pueden validar ambos contra el mismo stock inicial y dejar unidades negativas.

El script devuelve códigos simples. Un `-1` indica que el producto no existe, un `-2` indica stock insuficiente, y un valor positivo es el nuevo número de unidades cuando el descuento se realiza correctamente. La función traduce esos códigos a errores de negocio o a una respuesta con las unidades restantes.

Ejemplo de resultado de Query 15

```
1 {
2   "id_producto": "PRD004",
3   "unidades": 38
4 }
```

4.3 Queries que combinan MongoDB y Redis

4.3.1 Query 8: Stock de productos con menos de 50 unidades

La octava query expone el endpoint `GET /api/stock-bajo?umbral=50` y lista productos cuyo stock se encuentra por debajo del umbral indicado. Es una query polígota. Redis identifica rápidamente los productos con pocas unidades y MongoDB aporta la información descriptiva del catálogo.

En Redis se usa la clave `stock:unidades`, modelada como un Sorted Set. El identificador del producto es el valor del conjunto y la cantidad disponible es el score. La consulta usa un rango por score para obtener los productos con unidades menores al umbral, manteniendo el orden ascendente por stock.

Luego, con los identificadores devueltos por Redis, la función consulta la colección `productos` en MongoDB mediante `$in`. MongoDB devuelve los campos de catálogo, como nombre, categoría, precio unitario, vencimiento y proveedor. Finalmente, Node.js combina ambos resultados conservando el orden que vino de Redis.

Esta query justifica la separación entre catálogo y stock operativo. MongoDB no necesita actualizar un contador de unidades con cada operación, y Redis no necesita duplicar los datos descriptivos del producto.

Fragmento de resultado de Query 8

```
1 [
2   {
3     "_id": "PRD017",
4     "nombre": "Insulina canina",
5     "categoria": "Hormonal",
6     "precio_unit": 8500,
7     "vencimiento": "2026-08-15T00:00:00.000Z",
8     "proveedor": "MediAnimal",
9     "unidades": 15
10  },
11  {
12    "_id": "PRD015",
13    "nombre": "Praziquantel 50mg",
14    "categoria": "Antiparasitario",
15    "precio_unit": 1300,
16    "vencimiento": "2026-06-30T00:00:00.000Z",
17    "proveedor": "VetFarma SA",
18    "unidades": 20
19  }
20 ]
```

4.3.2 Query 14: Registro de nueva consulta médica

La decimocuarta query expone el endpoint `POST /api/consultas` y registra una nueva atención médica. Se apoya principalmente en MongoDB, usando las colecciones `pacientes`, `veterinarios` y `consultas`, pero también puede interactuar con Redis si la consulta informa productos consumidos.

Antes de insertar la consulta, la función valida que el paciente exista y esté activo. También valida que el veterinario exista y esté activo. Estas validaciones son necesarias porque `consultas` guarda referencias por identificador. Insertar una atención con un paciente o veterinario inexistente rompería la integridad referencial del modelo.

Si el request incluye `productos_usados`, la función consolida productos repetidos y valida que cada cantidad sea un entero positivo. Luego consulta Redis para comprobar que cada producto exista en `stock:unidades` y que haya unidades suficientes. La consolidación evita validar dos líneas del mismo producto como si fueran consumos independientes contra el mismo stock inicial.

Para el documento de la consulta, si el cliente no envía `_id`, la función genera el siguiente identificador con prefijo `CON`. Luego inserta la atención en MongoDB con fecha, tipo, motivo, diagnóstico, costo y estado. Una vez registrada la consulta, invalida las claves de caché que dependen de la colección `consultas`, que son `cache:top-diagnostics`, `cache:ingresos-vet-mes` y `cache:vets-consultas-60d`.

Cuando hay productos usados, el descuento se delega en la Query 15. De esta forma, la validación de stock y el decremento final usan la misma estructura de Redis y el descuento efectivo se realiza con el script Lua atómico.

Ejemplo de resultado de Query 14

```
1  {
2    "ok": true,
3    "consulta": {
4      "_id": "CON999",
5      "id_paciente": "P001",
6      "id_vet": "V001",
7      "fecha": "2026-06-06T00:00:00.000Z",
8      "tipo": "Consulta",
9      "motivo": "Control de rutina",
10     "diagnostico": "Sano",
11     "costo": 4500,
12     "estado": "Cerrada"
13   },
14   "stock": [
15     {
16       "id_producto": "PRD001",
17       "unidades": 118
18     }
19   ]
20 }
```

4.4 Queries cacheadas

Las queries cacheadas usan un patrón cache-aside. La aplicación primero consulta Redis con una clave determinada. Si el valor existe, lo deserializa desde JSON y devuelve ese resultado. Si no existe,

ejecuta la agregación correspondiente en MongoDB, serializa el resultado, lo guarda en Redis con un TTL y finalmente lo devuelve al cliente.

Este patrón se eligió porque MongoDB sigue siendo la fuente de verdad para los datos clínicos y administrativos. Redis solamente acelera lecturas frecuentes del dashboard. En consecuencia, los resultados cacheados son descartables, ya que si una clave expira o se borra la query vuelve a calcularse contra MongoDB.

Para hacer visible este comportamiento, el servidor agrega el header `X-Cache` en las respuestas cacheadas. El valor puede indicar `MISS` cuando se recalcula el resultado o `HIT` cuando se responde desde Redis. La implementación usa `AsyncLocalStorage` para asociar esos eventos de caché al request HTTP actual, evitando usar variables globales compartidas entre requests concurrentes.

4.4.1 Query 5: Veterinarios activos con consultas en los últimos 60 días

La quinta query expone el endpoint `GET /api/vets-activos-consultas-60d` y lista los veterinarios activos junto con la cantidad de consultas registradas en los últimos 60 días. Se calcula con MongoDB sobre las colecciones `veterinarios` y `consultas`, y el resultado se guarda temporalmente en Redis como caché.

La función toma la fecha del sistema y construye un rango de días calendario, desde el inicio del día ubicado 60 días atrás hasta el inicio del día siguiente al de ejecución. Este criterio es importante porque los datos del proyecto se cargan como fechas de día completo, no como instantes con hora clínica.

El pipeline comienza desde `veterinarios` con un `$match` sobre `activo` en `true`. Luego usa un `$lookup` con pipeline correlacionado contra `consultas`, comparando el veterinario de cada atención con el documento activo que se está procesando y filtrando la fecha dentro del rango calculado. En la proyección final, la cantidad se obtiene con `$size` sobre el arreglo de consultas recientes.

El resultado incluye también veterinarios activos sin consultas recientes, porque el objetivo es listar veterinarios activos y su cantidad de consultas. Finalmente se ordena por `cantidad_consultas_60d` en forma descendente y, ante empates, por apellido y nombre.

Esta query es buena candidata para caché porque cruza veterinarios con consultas y puede ejecutarse repetidamente desde el dashboard. Por eso se guarda en Redis bajo la clave `cache:vets-consultas-60d` con un TTL de 300 segundos. Cuando se registra una nueva consulta médica, la función de alta invalida esta clave para que el próximo pedido recalcule el resultado desde MongoDB.

Ejemplo de resultado de Query 5

```
1  [
2    {
3      "nombre": "Gabriela",
4      "apellido": "Fontana",
5      "matricula": "VET-0041",
6      "especialidad": "Clinica General",
7      "sucursal": "Palermo",
8      "activo": true,
9      "id_vet": "V001",
10     "cantidad_consultas_60d": 8
11   }
12 ]
```

4.4.2 Query 7: Top 5 diagnósticos más frecuentes

La séptima query expone el endpoint `GET /api/top-diagnostics` y obtiene los cinco diagnósticos más frecuentes registrados en las atenciones. Se calcula con MongoDB sobre la colección `consultas` y se guarda temporalmente en Redis como caché.

El pipeline agrupa los documentos por `diagnostico`, cuenta ocurrencias con `$sum`, ordena de mayor a menor por cantidad y limita el resultado a cinco elementos. La salida conserva el diagnóstico como identificador del grupo y agrega el campo `total`.

Como es una agregación que puede consultarse muchas veces desde el dashboard y no necesita recalcularse en cada lectura, el resultado se cachea en Redis bajo la clave `cache:top-diagnostics` con un TTL de 300 segundos. Cuando se registra una nueva consulta médica, la función de alta invalida esta clave para que el siguiente pedido vuelva a calcular el ranking desde MongoDB.

Ejemplo de resultado de Query 7

```
1  [
2    {
3      "_id": "Sano",
4      "total": 7
5    },
6    {
7      "_id": "Otitis externa",
8      "total": 5
9    },
10   {
11     "_id": "Dermatitis atopica",
12     "total": 4
13   },
14   {
15     "_id": "Infestacion parasitaria",
16     "total": 3
17   },
18   {
19     "_id": "Cirugia exitosa",
20     "total": 2
21   }
22 ]
```

4.4.3 Query 11: Ingresos por veterinario en el mes actual

La undécima query expone el endpoint `GET /api/ingresos-vet-mes`. Su resultado se obtiene leyendo una vista formal de MongoDB llamada `vista_ingresos_vet_mes`, definida sobre la colección `consultas`.

El pipeline de la vista calcula el rango del mes actual dentro de MongoDB usando la variable `$$NOW`. A partir de ese valor, filtra las consultas cuya fecha pertenece al mes en curso, agrupa por veterinario, suma el campo `costo` como ingresos y cuenta cuántas atenciones componen ese total.

Luego, la vista cruza el resultado con `veterinarios` para incorporar el nombre completo del profesional y su sucursal. La salida queda compuesta por el identificador del veterinario, el nombre del veterinario, la sucursal, el total de ingresos y la cantidad de atenciones. El resultado se ordena por ingresos

descendientes.

El endpoint guarda en Redis durante 60 segundos el resultado de leer la vista, porque las vistas estándar de MongoDB se calculan al consultarse y no almacenan el resultado materializado.

Ejemplo de resultado de Query 11

```
1  [  
2    {  
3      "ingresos": 25000,  
4      "cantidad": 1,  
5      "id_vet": "V006",  
6      "veterinario": "Mateo Bianchi",  
7      "sucursal": "Recoleta"  
8    }  
9  ]
```


5 Limitaciones

La principal limitación técnica es que MongoDB y Redis no comparten una transacción global. El caso más sensible aparece al registrar una consulta que consume productos, donde el sistema inserta la atención en MongoDB y descuenta unidades en Redis. Para reducir el riesgo de inconsistencias, primero se valida el stock disponible, luego se inserta la consulta y finalmente se descuenta con un script Lua atómico. Aun así, si hubiera una falla justo entre ambos motores, podría existir una inconsistencia temporal.

Otra limitación es que la caché es best-effort. Las claves `cache:*` tienen TTL corto y se invalidan cuando la aplicación registra una nueva consulta, pero podrían quedar datos levemente desactualizados si alguien modifica MongoDB por fuera de la API. Esto es aceptable para consultas de dashboard, porque la caché no se usa como fuente de verdad ni para decisiones transaccionales.

Redis se usa como almacenamiento operativo del stock. En este setup corre dentro de Docker con la configuración simple del proyecto. Para producción habría que endurecer su persistencia y tolerancia a fallos, por ejemplo configurando AOF o snapshots RDB según la necesidad, replicación, backups y una política clara de recuperación ante caídas.

También se simplificó la seguridad del entorno. MongoDB y Redis se levantan sin autenticación ni TLS para facilitar la ejecución local. En un ambiente real deberían usarse credenciales, secretos fuera del repositorio, redes internas, firewall y conexiones seguras.

6 Conclusiones

El trabajo permitió aplicar una arquitectura de persistencia polígloa con dos motores NoSQL de paradigmas distintos y responsabilidades bien separadas. MongoDB funcionó como base principal del dominio clínico y administrativo, mientras que Redis se usó para stock operativo, decrementos atómicos y caché de consultas frecuentes.

El modelo de datos se diseñó a partir de las consultas que debía resolver el sistema. Las entidades principales se mantuvieron como colecciones independientes y relacionadas por identificadores, lo que permitió resolver los cruces necesarios con pipelines de agregación. Redis se utilizó solamente donde aportaba una ventaja concreta, como ordenar y filtrar stock por cantidad, descontar unidades de forma atómica y acelerar lecturas repetidas.

La solución final resuelve las quince consultas y servicios definidos, mantiene el código organizado en módulos separados y deja documentadas las decisiones de diseño principales. Si bien existen limitaciones propias de coordinar dos motores sin transacción global, la implementación resulta adecuada para el alcance planteado, ya que es simple de levantar, reproducible, fácil de probar y coherente con el uso de MongoDB y Redis dentro del sistema VetSalud.